Security in Software Applications

Secure Software Design

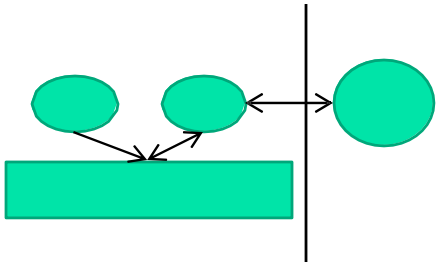
SAPIENZA
UNIVERSITÀ DI ROMA

Daniele Friolo

friolo@di.uniroma1.it

https://danielefriolo.github.io

What is design?



- ISO/IEC 12207:2008:
  - "Software architectural design.
    - For each software item …the implementer shall transform the requirements for the software item into an architecture that describes its top-level structure and identifies the software components.
    - It shall be ensured that all the requirements for the software item are allocated to its software components and further refined to facilitate detailed design."
  - "Software detailed design process is to provide a design for the software that implements and can be verified against the requirements…"
- In short:
  - Determine *components* (e.g., classes, programming languages, frameworks, database systems, etc.) that you'll use, and how you'll use them (*interconnections/APIs*), to *solve the problem*

Developing secure software is *not* new or unknown

- Saltzer & Schroeder principles, and many specifics, publicly known since at least 1970s
- Problem is that most software *developers* do not know how to do it
  - Attackers, of course, know this
- Knowing various principles and rules-of-thumb ahead-of-time can avoid many problems later

Saltzer and Schroeder design principles (1)

- **Least privilege**
  - Each user and program should operate using fewest privileges possible
  - Limits the damage from an accident, error, or attack
  - Reduces number of potential interactions among privileged programs
    - Unintentional, unwanted, or improper uses of privilege less likely
  - Extend to the internals of a program: only smallest portion of program which needs those privileges should have them
- **Economy of mechanism/Simplicity**
  - Protection system's design should be simple and small as possible
  - "techniques such as line-by-line inspection of software and physical examination of hardware that implements protection mechanisms are necessary. For such techniques to be successful, a small and simple design is essential.'"
  - Aka "KISS" principle («keep it simple, stupid")

Saltzer and Schroeder design principles (2)

- **Open design**
  - The protection mechanism must not depend on attacker ignorance
  - Instead, the mechanism should be public
    - Depend on secrecy of relatively few (and easily changeable) items like passwords or private keys
  - An open design makes extensive public scrutiny possible
  - Makes it possible for users to convince themselves system is adequate
  - Not realistic to maintain secrecy for distributed system
    - Decompilers/subverted hardware quickly expose implementation “secrets”
    - Even if you pretend that source code is necessary to find exploits (it isn't), source code has often been stolen and redistributed
    - This is one of the oldest and strongly supported principles, based on many years in cryptography
      - Kerckhoffs's Law: “A cryptosystem should be designed to be secure if everything is known about it except the key information”
      - Claude Shannon (inventor of information theory) restated Kerckhoff's Law as: “[Assume] the enemy knows the system”

Saltzer and Schroeder design principles (3)

- **Complete mediation** ("non-bypassable")
  - Every access attempt must be checked; position the mechanism so it cannot be subverted. For example, in a client-server model, generally the server must do all access checking because users can build or modify their own clients
- **Fail-safe defaults** (e.g., permission-based approach)
  - The default should be denial of service, and the protection scheme should then identify conditions under which access is permitted
  - More generally, installation should be secure by default
- **Separation of privilege**
  - Ideally, access to objects should depend on more than one condition, so that defeating one protection system won't enable complete access

Saltzer and Schroeder design principles (3)

- **Least common mechanism**
  - Minimize the amount and use of shared mechanisms (e.g. use of the /tmp or /var/tmp directories)
  - Shared objects provide potentially dangerous channels for information flow and unintended interactions
- **Psychological acceptability / Easy to use**
  - The human interface must be designed for ease of use so users will routinely and automatically use the protection mechanisms correctly
  - Mistakes will be reduced if the security mechanisms closely match the user's mental image of his or her protection goals

Design principles…

- Any design must also consider other requirements and specific circumstances
- Not all details apply to all systems
  - Web apps ≠ Local apps ≠ GUI ≠ setuid programs
  - But better to know the larger set of issues

NIST SP 800-160 volume 1 Appendix F (design principles)

Security Architecture and Design	
Clear Abstractions	Hierarchical Trust
Least Common Mechanism	Inverse Modification Threshold
Modularity and Layering	Hierarchical Protection
Partially Ordered Dependencies	Minimized Security Elements
Efficiently Mediated Access	Least Privilege
Minimized Sharing	Predicate Permission
Reduced Complexity	Self-Reliant Trustworthiness
Secure Evolvability	Secure Distributed Composition
Trusted Components	Trusted Communication Channels
Security Capability and Intrinsic Behaviors	
Continuous Protection	Secure Failure and Recovery
Secure Metadata Management	Economic Security
Self-Analysis	Performance Security
Accountability and Traceability	Human Factored Security
Secure Defaults	Acceptable Security
Life Cycle Security	
Repeatable and Documented Procedures	Secure System Modification
Procedural Rigor	Sufficient Documentation

Analysis approaches

- **Attacker-centric**
  - Starts with an attacker
  - Evaluates their goals and how they might achieve them (e.g., via entry points)
  - Used in CERT attack modeling approach
- **Design-centric**
  - Starts with the design of the system: steps through system model, looking for types of attacks against each element
  - Used in threat modeling in Microsoft's SecurityDevelopment Lifecycle
- **Asset-centric**
  - Starts from key assets entrusted to a system

Microsoft STRIDE Approach

- Decompose system into relevant (design) components
  - Uses simple data flow diagrams, identify trust boundary
- Analyze each component for susceptibility to threats
- Mitigate the threats
- Now part of the Microsoft SDL with the Thread Modeling Tool
  - https://www.microsoft.com/en-us/securityengineering/sdl/threatmodeling

"STRIDE" Threat	Security Property
Spoofing	Authentication
Tampering	Integrity
Repudiation	Non-repudiation
Information disclosure	Confidentiality
Denial of service	Availability
Elevation of privilege	Authorization

CERT Attack Modeling

- Create simple design overview
- Develop "*attack trees*" – top is attacker's objective, decompose down into conditions that achieve that (with AND or OR)
- Devise methods to counterattack (build on attack patterns)
- E.G., Buffer Overflow Attack Pattern:
  - *Goal*: Exploit buffer overflow vulnerability to perform malicious function on target system
  - *Precondition*: Attacker can execute certain programs on target system
  - Attack:

AND:

1. Identify executable program on target system susceptible to buffer overflow vulnerability
2. Identify code that will perform malicious function when it executes with program's privilege
3. Construct input value that will force code to be in program's address space
4. Execute program in a way that makes it jump to address at which code resides

  - *Postcondition*: Target system performs malicious function

More on attack/threat modeling:

https://owasp.org/www-community/Threat_Modeling_Process

https://insights.sei.cmu.edu/blog/threat-modeling-12-available-methods/

Conclusions

- There are various key design principles
  - E.G., Saltzer and Schroeder
- Need to design program to counterattack, e.g.:
  - Minimize privileges
  - Counter TOCTOU issues
- Use attack/threat modeling to look for potentially-successful attacks
  - Before the attacker tries them
- Many design approaches for self-protection
- Consider principles and rules-of-thumb in design